

```
37         self.file = open(os.path.join(settings.job_dir, fingerprint), 'a')
38         self.file.seek(0)
39         self.fingerprints.update({fingerprint: self.file})
40
41
42     @classmethod
43     def from_settings(cls, settings):
44         debug = settings.getbool('SUPERFUTUR_DEBUG')
45         return cls(job_dir(settings), debug)
46
47     def request_seen(self, request):
48         fp = self.request_fingerprint(request)
49         if fp in self.fingerprints:
50             return True
51         self.fingerprints.add(fp)
52         if self.file:
53             self.file.write(fp + os.linesep)
```

Software Engineering 2 – Vorlesung

Christian Macho

Software Engineering Research Group

Universität Klagenfurt



<https://mitschi.github.io/>



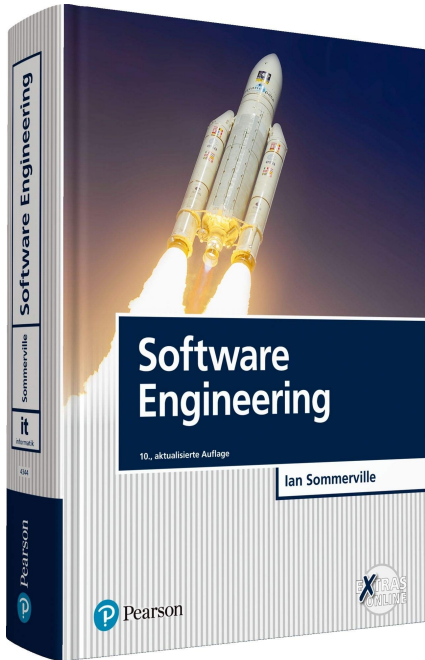
@Mitschiii



christian.macho@aau.at

EINFÜHRUNG SE

Literatur



Software Engineering,
Ian Sommerville,
10. Auflage

Software Engineering

Software Engineering

Software Engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the **early stages** of system specification through to maintaining the system **after it has gone into use**. [1]

Software Engineering

Software Engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the **early stages** of system specification through to maintaining the system **after it has gone into use**. [1]

- Technische Disziplin
 - Bewusste Selektion von Theorien, Methoden, Werkzeugen
 - Neue Lösungen, auch unter organisatorischen und finanziellen Einschränkungen
- Alle Aspekte
 - Technische Aspekte
 - Projektmanagement
 - Entwicklung von Werkzeugen, Methoden und Theorien zur Unterstützung der professionellen Software Entwicklung

Software Engineering

Software Engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the **early stages** of system specification through to maintaining the system **after it has gone into use**. [1]

- Technische Disziplin

- Bewusste Selektion von Theorien, Methoden, Werkzeugen
- Neue Lösungen, auch unter organisatorischen und finanziellen Einschränkungen

- Aspekte

- Technische Aspekte
- Projektmanagement
- Entwicklung von Werkzeugen, Methoden und Theorien zur Unterstützung der professionellen Software Entwicklung

Software Engineering

Software Engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the **early stages** of system specification through to maintaining the system **after it has gone into use**. [1]

- **Warum brauchen wir Software Engineering?**
 - Um zuverlässige und vertrauenswürdige Systeme wirtschaftlich und schnell herzustellen.
- **Unterschied zur Informatik?**
 - Theorien und Methoden der SW Systeme vs. Probleme der SW Herstellung
- **Unterschied von Software Projekten auf der Uni vs. im „echten“ Leben?**

“Hacken” vs. Software Engineering

Personal Software	Industrial-strength Software
Developer is user	Client is user
Bugs are tolerable	Bugs are not tolerated
UI not important	UI important
No/Minor documentation	Lots of documentation
SW not in critical use	Supports business functions
Reliability/Robustness not crucial	Reliability/Robustness is crucial
No investment	Heavy investment (5\$-25\$ per LOC)
Portability not so important	Portability is an economic advantage

FAQ

1. Was ist das Besondere an Software?
2. Wozu braucht man Software Engineering?
3. Was sind Merkmale guter Software?
4. Was sind die Herausforderungen im Software Engineering?



HERAUSFORDERUNGEN UND AKTIVITÄTEN IM SE

1. Was ist besonders?

SW ...

- ... ist unsichtbar
- ... verbraucht keinen Platz
- ... hat keine physikalische Substanz
- ... riecht nicht
- ... hat kein Aussehen

ABER

SW ...

- ... ist risikobehaftet
- ... benötigt Platz und Zeit (Entwicklung)
- ... lebt länger, als man denkt
- ... kann großen Schaden verursachen

1. Was ist besonders?

SW ...

- ... ist unsichtbar
- ... verbraucht keinen Platz
- ... hat keine physikalische Substanz
- ... riecht nicht
- ... hat kein Aussehen

ABER

SW ...

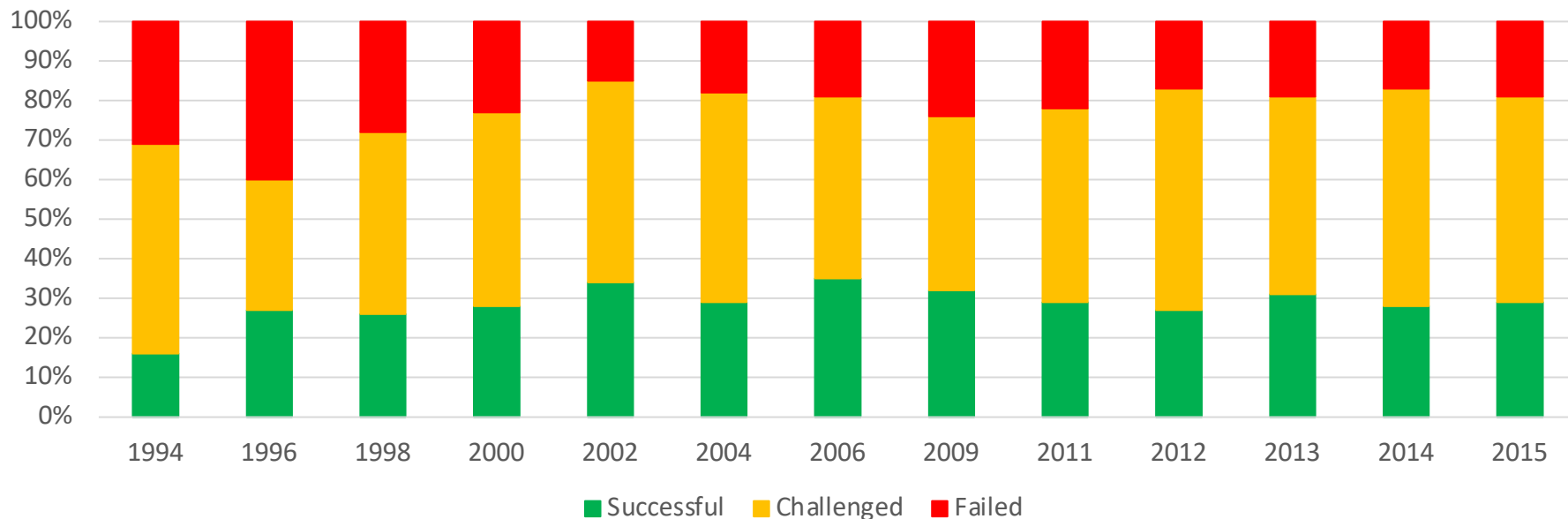
- ... ist risikobehaftet
- ... benötigt Platz und Zeit (Entwicklung)
- ... lebt länger, als man denkt
- ... kann großen Schaden verursachen

2. Wozu Software Engineering?

- Wieviele SW-Projekte gehen schief?
- Wieviele SW-Projekte sind “in-time”?
- Wieviele SW-Projekte sind “on-budget”?

2. Wozu Software Engineering?

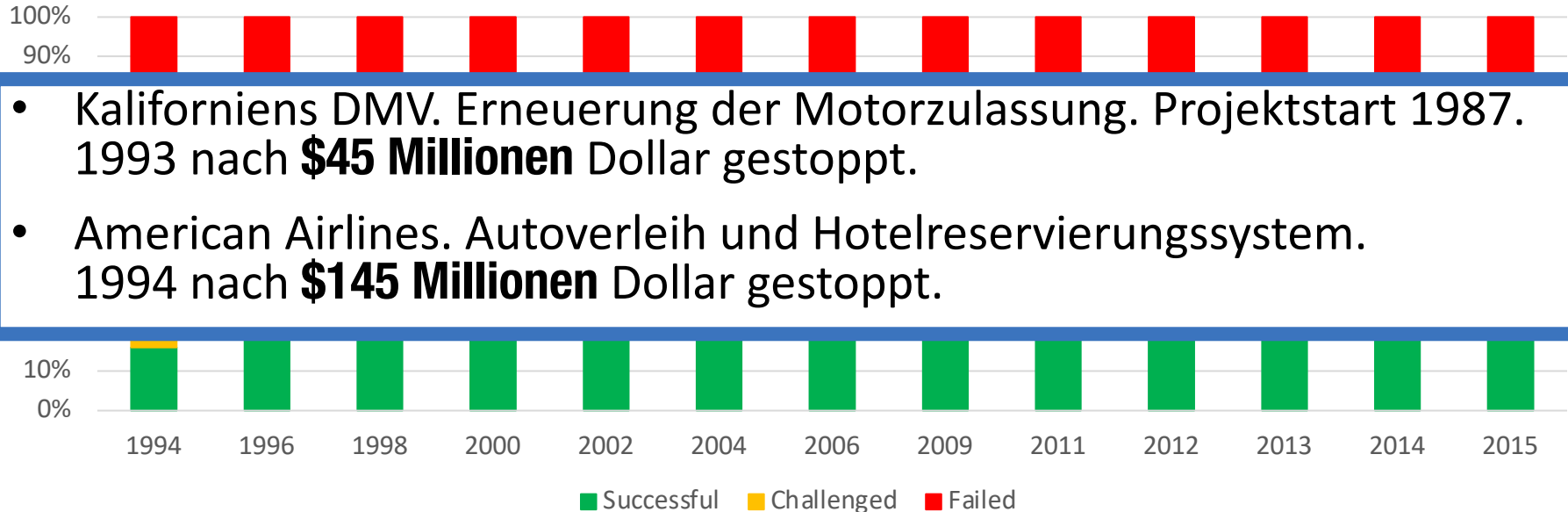
Chaos Report – Standish Group



https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf

2. Wozu Software Engineering?

Chaos Report – Standish Group



https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf

2. Wozu Software Engineering?



**Terac-25 Strahlentherapie,
1985 – 1987**

2. Wozu Software Engineering?



**Terac-25 Strahlentherapie,
1985 – 1987**



Ariane 5, 1996

2. Wozu Software Engineering?



**Terac-25 Strahlentherapie,
1985 – 1987**



Ariane 5, 1996



**Flughafen Heathrow,
2018**

3. Was sind Merkmale guter Software?

3. Was sind Merkmale guter Software?

Good software should deliver the **required functionality and performance** to the user and should be **maintainable, dependable, and usable** [1].



1. Wartbarkeit



**2. Sicherheit und
Zuverlässigkeit**

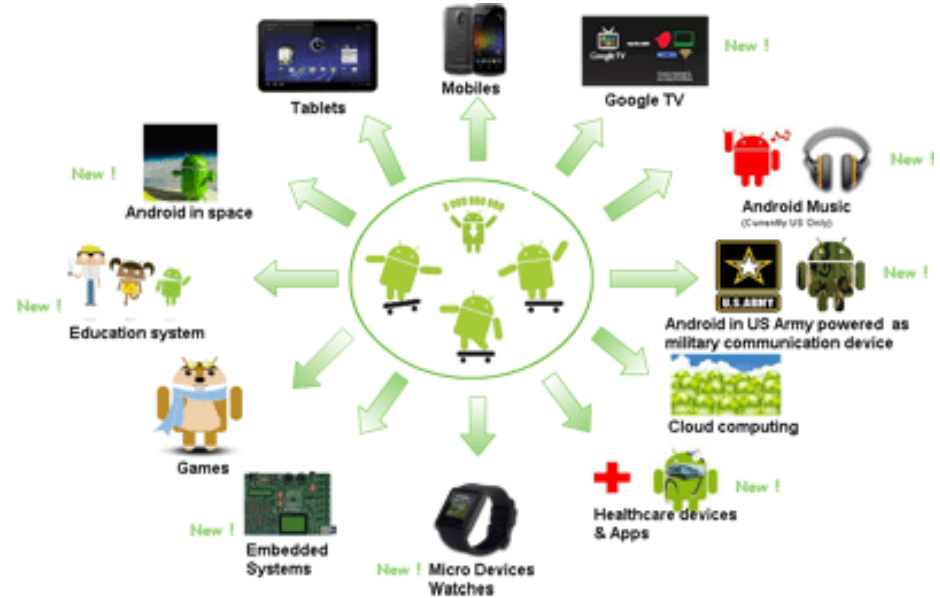


3. Effizienz



**4. Benutzerfreund-
lichkeit (Acceptability)**

4. Herausforderungen im SE?



Heterogenität

4. Herausforderungen im SE?



Neue Technologien,
wirtschaftlicher und sozialer Wandel

4. Herausforderungen im SE?



Sicherheit und Zuverlässigkeit

SOFTWAREENTWICKLUNGSPROZESSE I

Wie Software entwickeln?

- Wie entwickeln Sie Software?
- Wie gehen Sie dabei vor?



Herausforderungen

- Unterschiedliche Dimensionen der Komplexität
 - Eine Aufgabe **selbst zum ersten Mal** erledigen
 - Eine Aufgabe **überhaupt zum ersten Mal** erledigen
 - Wenn das **Problemfeld unklar** ist
 - Wenn die **Problemlösung umfangreich** ist
 - Wenn das **Problem facettenreich** ist
 - Wenn die **Facetten des Problems ineinandergreifen** (und damit evtl. Teillösungen ineinandergreifen)
- Möglichkeiten, um komplexe Aufgaben zu lösen
 - Einen Weg **ausprobieren** und sehen, ob er funktioniert
 - Jemanden anderen eine **Lösung finden lassen**
 - Planen, Ausführen, Überprüfen und Anpassen (PDCA)



Herausforderungen

- Naives Grundmodell: Code & Fix
 - Schreibe ein Programm
 - Finde und behebe die Fehler im Programm

Herausforderungen

- Naives Grundmodell: Code & Fix
 - Schreibe ein Programm
 - Finde und behebe die Fehler im Programm

+ Schnell lauffähiges Programm
+ Coding und spontanes Testen sind einfach

- Projekt nicht planbar
- Anforderungen werden nicht erhoben
- Keine Basis für Tests
- Schwierige und teure Wartung
- Wie wird erworbenes Wissen transferiert?
- Skalierbarkeit?

Herausforderungen

- Naives Grundmodell: Code & Fix
 - Schreibe ein Programm

Softwareentwicklungsprozesse (bzw. Vorgehensmodelle) helfen!

+ Coding und spontanes Testen sind einfach

- Keine Basis für Tests
- Schwierige und teure Wartung
- Wie wird erworbenes Wissen transferiert?
- Skalierbarkeit?

Aktivitäten

Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- Wozu?
 - Überblick behalten
 - Effektives und effizientes Arbeiten im Team
 - Gemeinsame Notation, konsistente Bedeutung



Aktivitäten

Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- **Aktivitäten**



Aktivitäten

Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- **Aktivitäten**
 - **Spezifizierung:** Festlegung der Funktionalität und Einschränkungen

Aktivitäten

Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- **Aktivitäten**
 - **Spezifizierung:** Festlegung der Funktionalität und Einschränkungen
 - **Design und Implementierung:** Umsetzung der Architektur und Entwicklung des Systems

Aktivitäten

Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- **Aktivitäten**
 - **Spezifizierung:** Festlegung der Funktionalität und Einschränkungen
 - **Design und Implementierung:** Umsetzung der Architektur und Entwicklung des Systems
 - **Validierung/Verifizierung:** Überprüfung auf Erfüllung der Anforderungen des Kunden und der korrekten Funktionalität

Aktivitäten

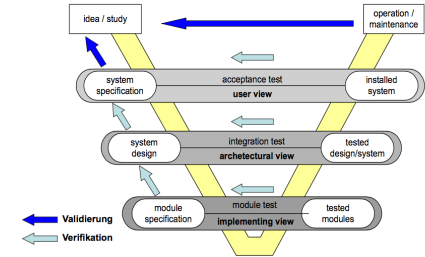
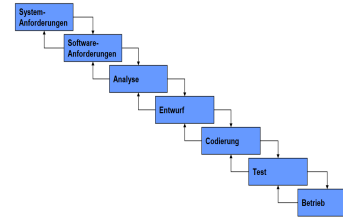
Ein SW-Prozess Modell ist eine **abstrakte Repräsentation** einer Menge an **Aktivitäten und Leistungen**, die zur **Erzeugung eines SW-Produkts** führen.

- **Aktivitäten**
 - **Spezifizierung:** Festlegung der Funktionalität und Einschränkungen
 - **Design und Implementierung:** Umsetzung der Architektur und Entwicklung des Systems
 - **Validierung/Verifizierung:** Überprüfung auf Erfüllung der Anforderungen des Kunden und der korrekten Funktionalität
 - **Evolution:** Anpassungen und Erweiterungen aufgrund von veränderten Kundenwünschen

Softwareentwicklungsprozesse

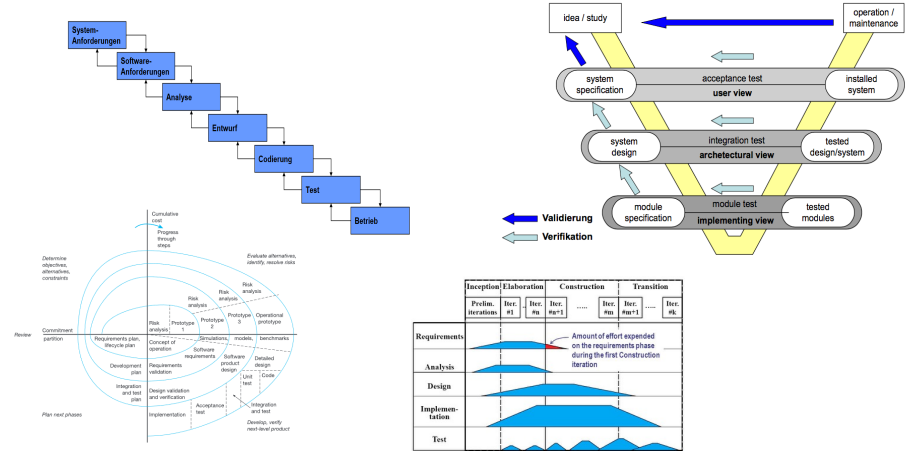
Softwareentwicklungsprozesse

- Sequentielle Modelle
 - Wasserfallmodell
 - V-Modell



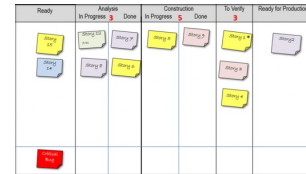
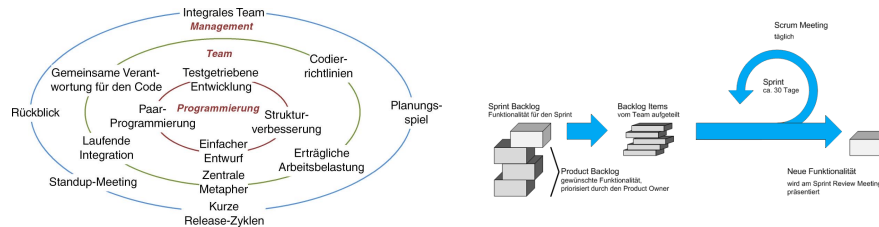
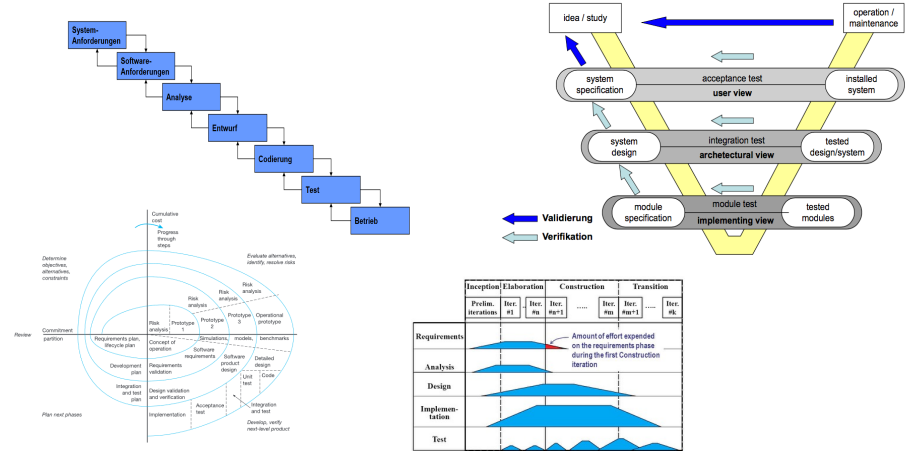
Softwareentwicklungsprozesse

- Sequentielle Modelle
 - Wasserfallmodell
 - V-Modell
- Iterative Modelle
 - Spiralmodell
 - ~~(Rational Unified Process – RUP)~~



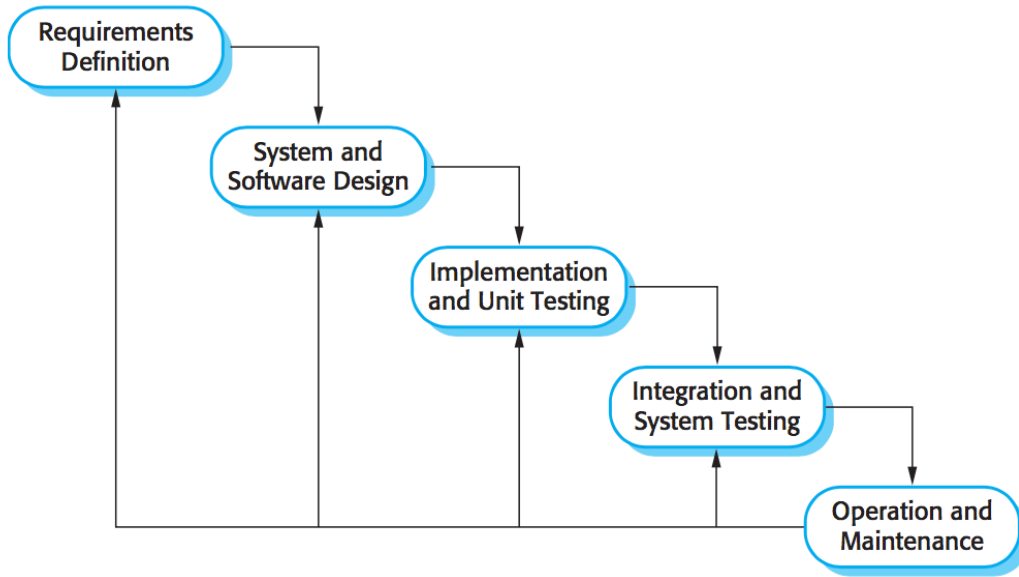
Softwareentwicklungsprozesse

- Sequentielle Modelle
 - Wasserfallmodell
 - V-Modell
- Iterative Modelle
 - Spiralmodell
 - ~~(Rational Unified Process – RUP)~~
- Agile Methoden
 - XP
 - Scrum
 - Kanban
 - Scaled Agile



SEQUENTIELLE PROZESSE

Das Wasserfall Modell



Royce, 1970

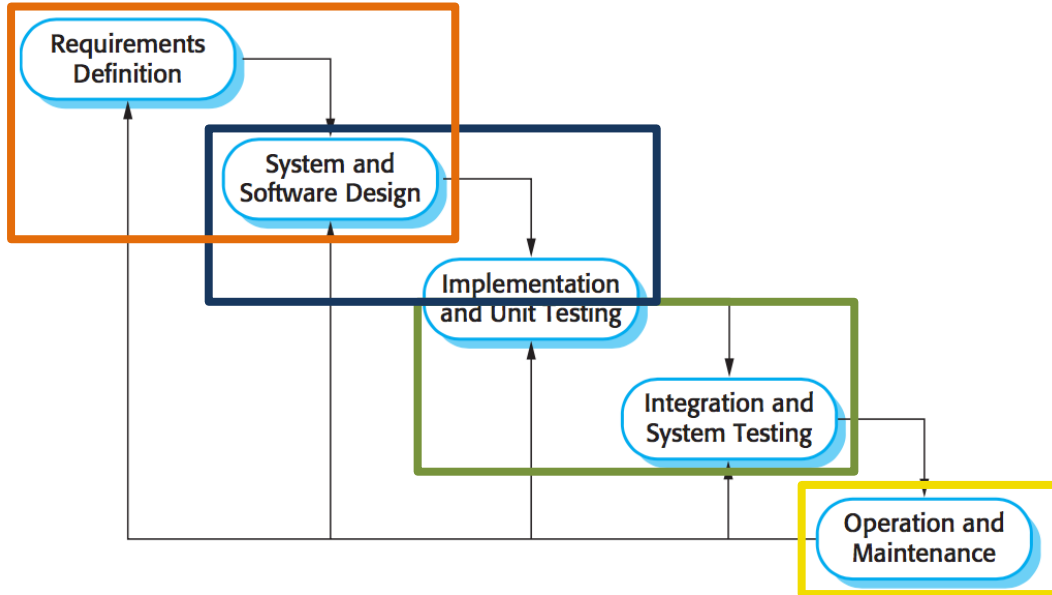
Charakteristika

- Top-Down
- Festgelegte Reihenfolge der Aktivitäten
- Eingeschränkte Rückkopplung
- Zwischenprodukt am Ende jeder Aktivität
- Benutzerbeteiligung nur zu Beginn
- Variante: formale Systementwicklung

Das Wasserfall Modell

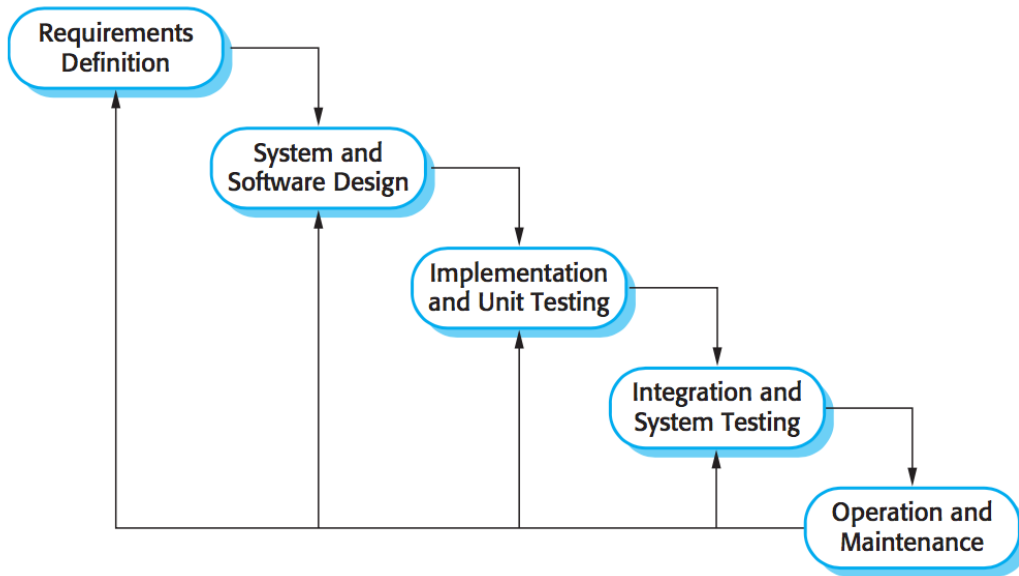
Charakteristika

- Top-Down
- Festgelegte Reihenfolge der Aktivitäten
- Eingeschränkte Rückkopplung
- Zwischenprodukt am Ende jeder Aktivität
- Benutzerbeteiligung nur zu Beginn
- Variante: formale Systementwicklung



Royce, 1970

Das Wasserfall Modell

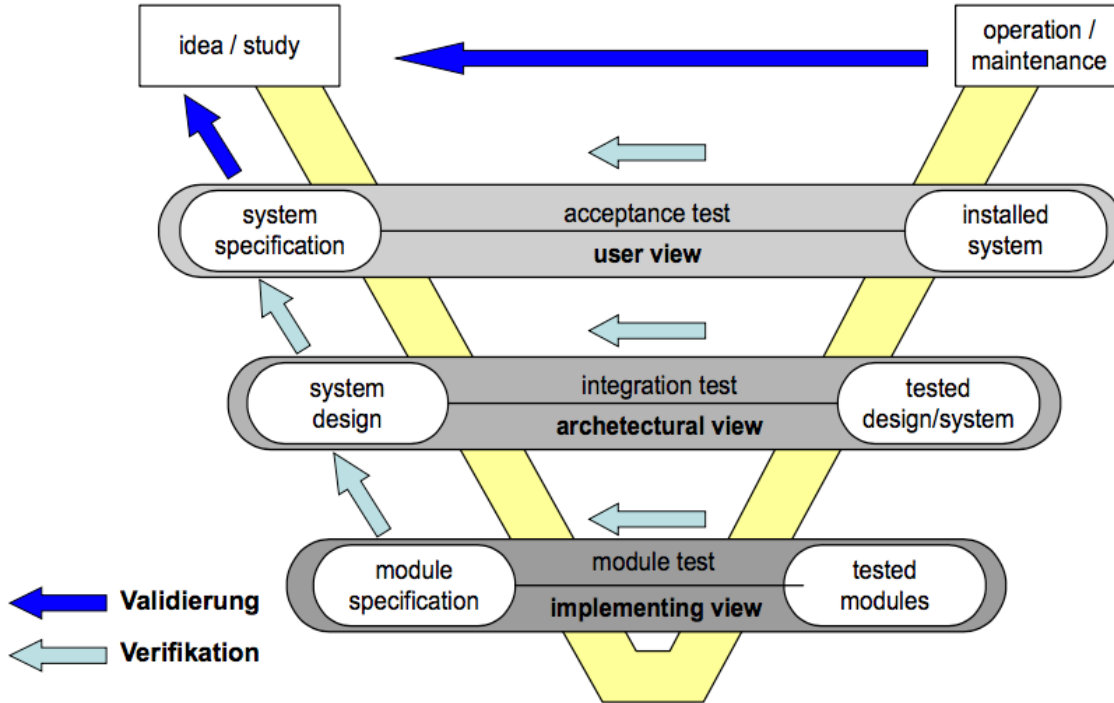


Royce, 1970

- + (Theoretisch) gut planbar
- + Geringer Managementaufwand

- Notwendige „Kurskorrekturen“ nicht frühzeitig erkennbar
- Sequentialität nicht immer nötig
- Gefahr, dass Dokumente wichtiger als das System werden
- Risikofaktoren werden u.U. zu wenig berücksichtigt

Das V – Modell

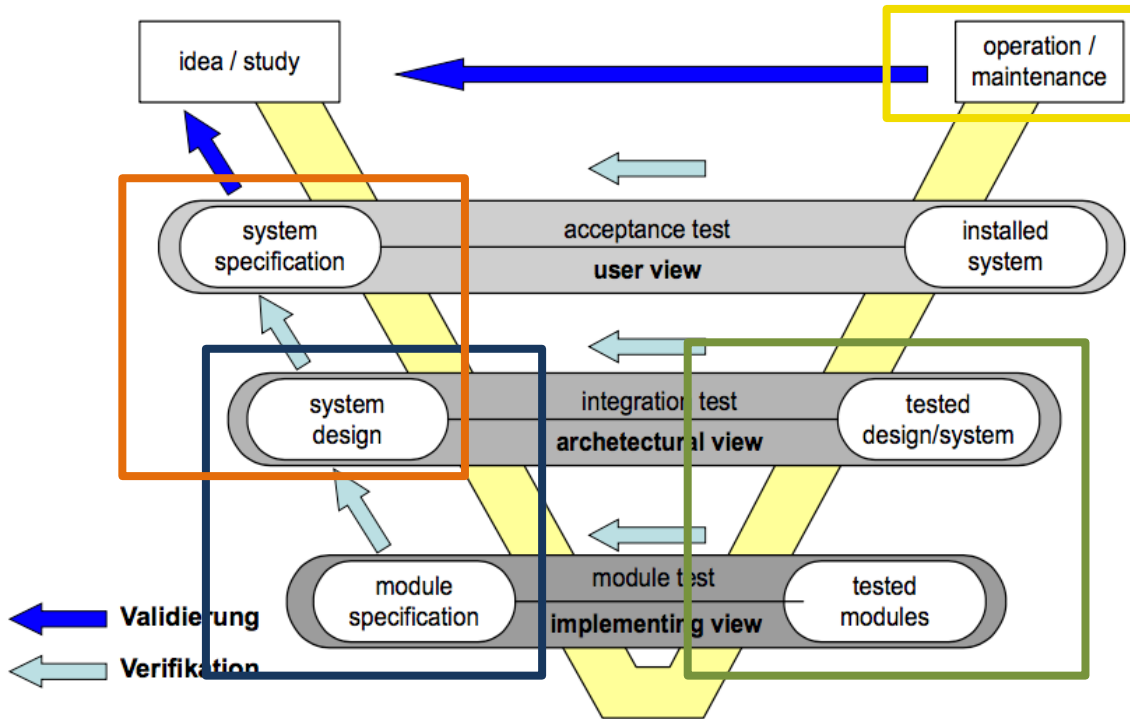


Charakteristika

- Qualitätssicherung integriert
- Sehr umfangreiches Modell, wurde für große Projekte konzipiert
- Phasen, Aktivitäten, (Zwischen-Produkte)
- Verifikation und Validation
- Produkt muss mit den Anforderungen und den Kundenwünschen übereinstimmen

[QS-Folien, TU Wien]

Das V – Modell

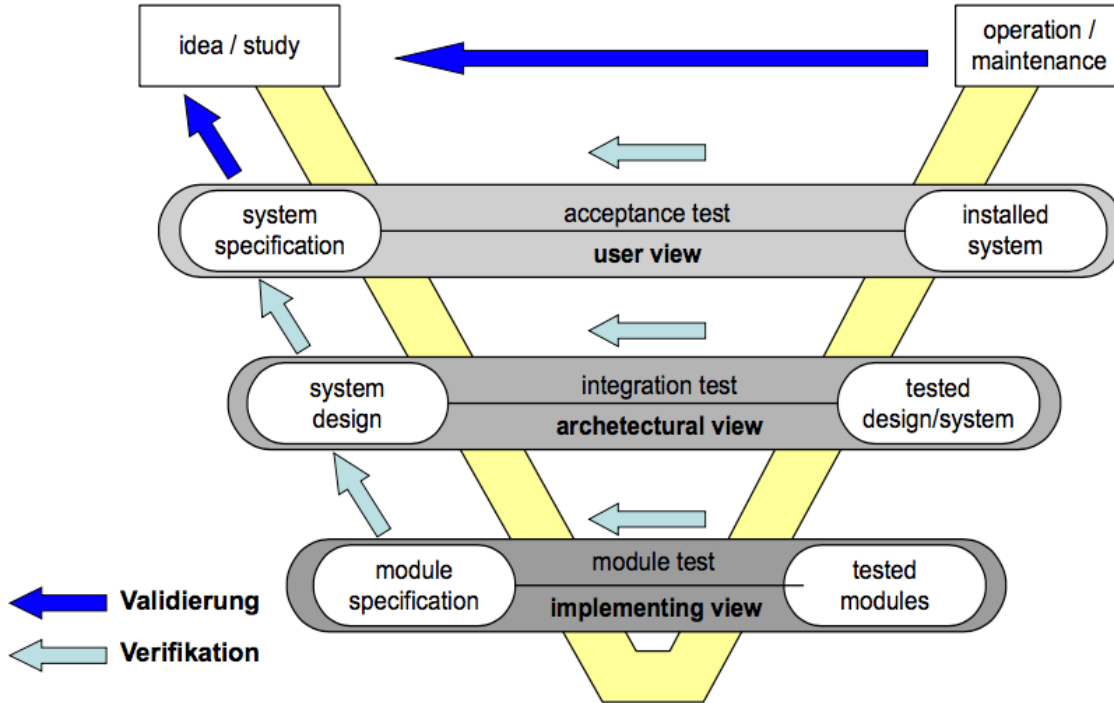


[QS-Folien, TU Wien]

Charakteristika

- Qualitätssicherung integriert
- Sehr umfangreiches Modell, wurde für große Projekte konzipiert
- Phasen, Aktivitäten, (Zwischen-Produkte)
- Verifikation und Validation
- Produkt muss mit den Anforderungen und den Kundenwünschen übereinstimmen

Das V – Modell



Anwendung

- Gut anwendbar bei klarer, relativ fixer Funktionalität
- System- und Basissoftware, wie OS, DB, Web-Server
- Branchen Software wie SAP R/3

[QS-Folien, TU Wien]

Das V – Modell

- + Integrierte, detaillierte Beschreibung von Systemerstellung, Qualitätssicherung, Konfigurationsmanagement und Projektmanagement
- + Generisches Vorgehensmodell
- + Lässt sich gut anpassen und erweitern
- + Gut geeignet für große Projekte
- + Einfach durchzuführen
- + Schränkt Freiheitsgrade stark ein, daher auch für sehr große Projekte anwendbar
- + Sehr effizient bei bekannten und konstanten Anforderungen

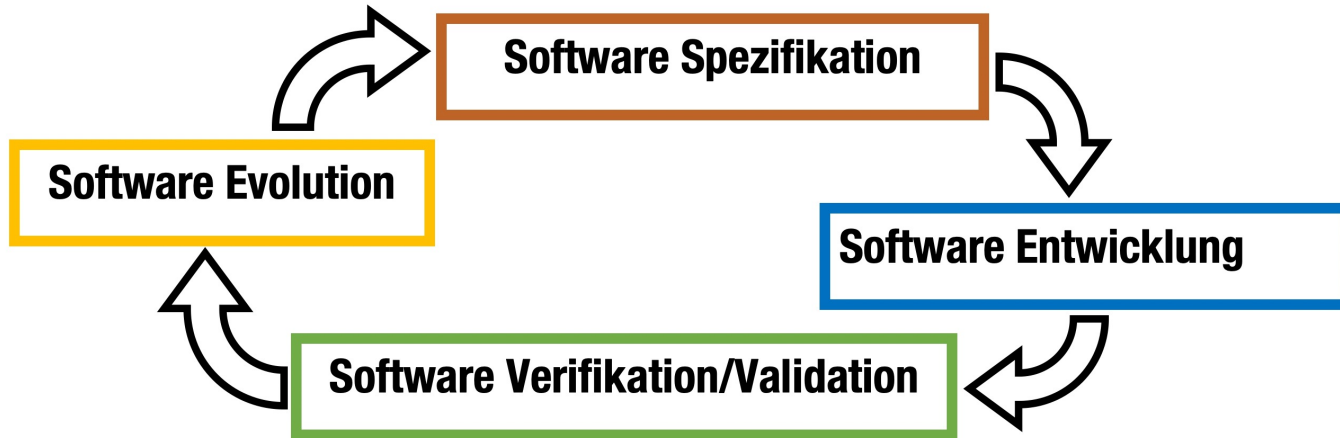
- Software-Bürokratie bei kleinen & mittleren Projekten
- Risiken gesammelt am Schluss —> „Big Bang“
- Starr während des Ablaufs

Änderungen?



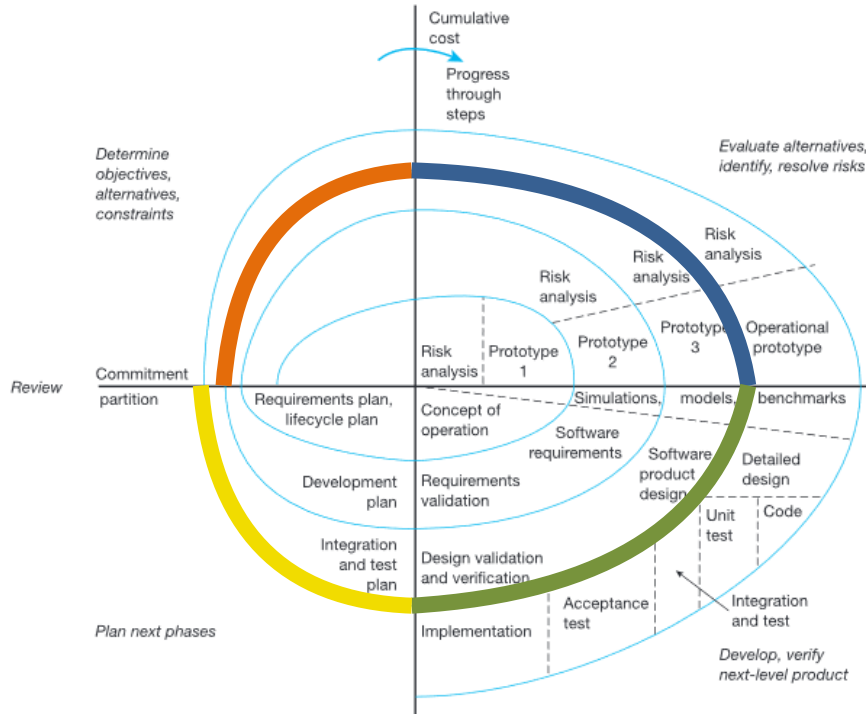
ITERATIVE PROZESSE

Iterative Prozessmodelle



Weiterentwicklung der sequentiellen Methoden
unterstützen inkrementelle Entwicklung

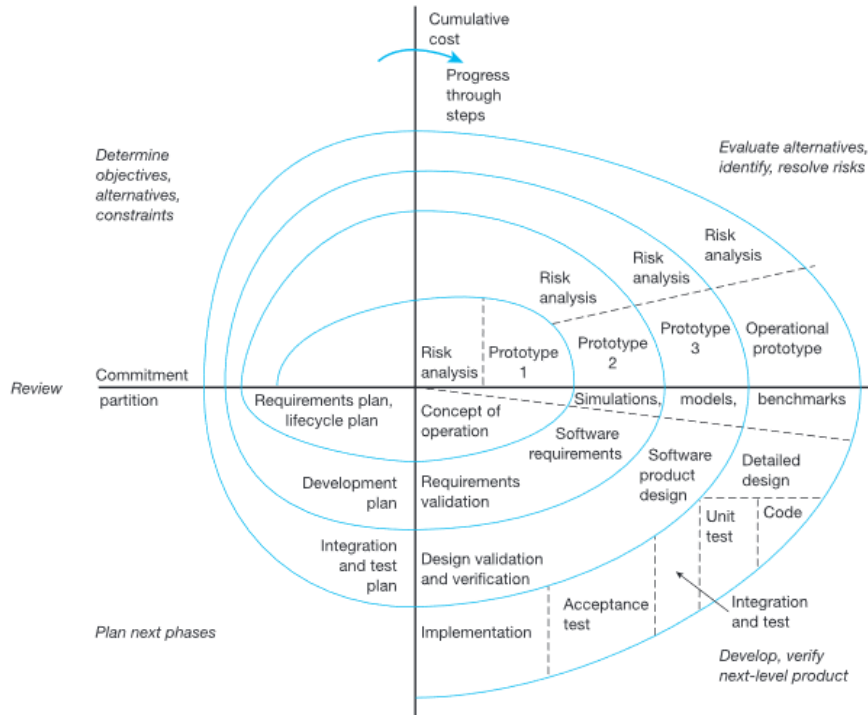
Das Spiral Modell



Charakteristika

- Rundenbasiert
 - **Entwicklungsziele** und Einschränkungen
 - Risiko-Analyse und Alternativen, Prototypen
 - **Entwicklungsphase** (Design, Programmierung, Verifikation, Validierung)
 - **Planungsphase** (für den nächsten Iterationsschritt)
- Gut für große Systeme, die das Zusammenarbeiten von mehreren Disziplinen erfordert

Das Spiral Modell



- + Risiken können früher erkannt werden
- + Volatile Anforderungen können besser berücksichtigt werden
- + Inkrementelle Auslieferung wird erleichtert
- + Anpassungsfähig!

- Mehrarbeit
- Komplexeres Projektmanagement
- Theoretisches Modell

AGILE PROZESSE

Agile Prozessmodelle

„Agil“ wird hier im Sinne von „bereit sich zu bewegen“, „aktiv“ oder „geschickt“ verstanden

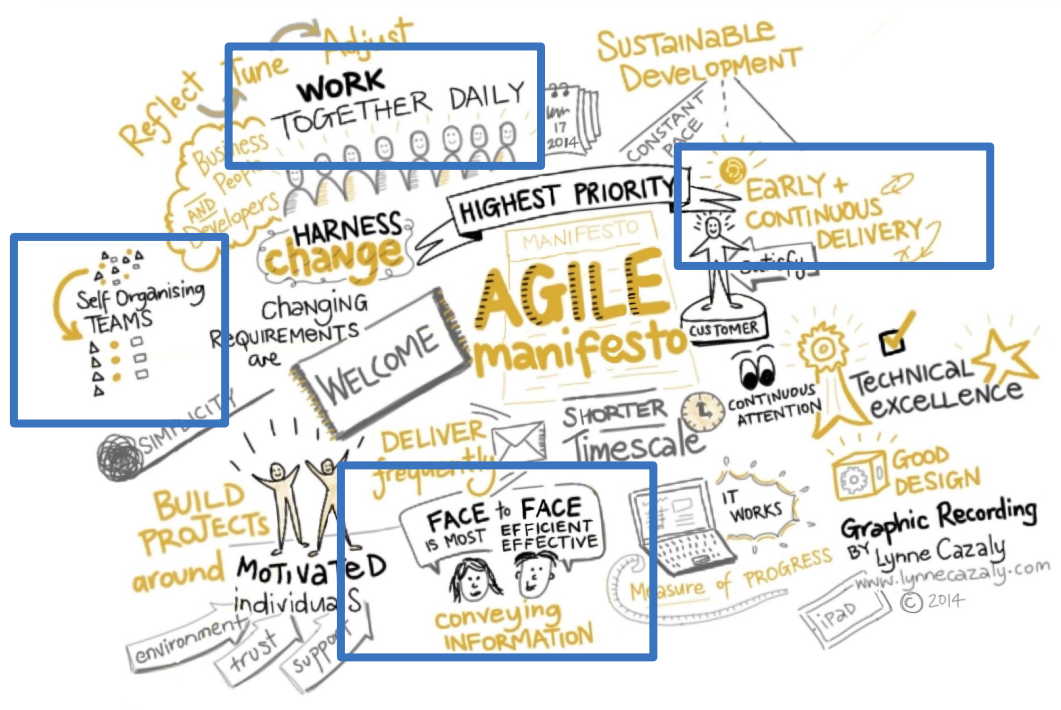
- Gegenbewegung **“mit Anti-Prozessen”**

- XP, SCRUM, Adaptive Software Development, FDD

Agiles Manifest: Werte und Prinzipien

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

Das Agile Manifesto



Agile Prozessmodelle

Charakteristika

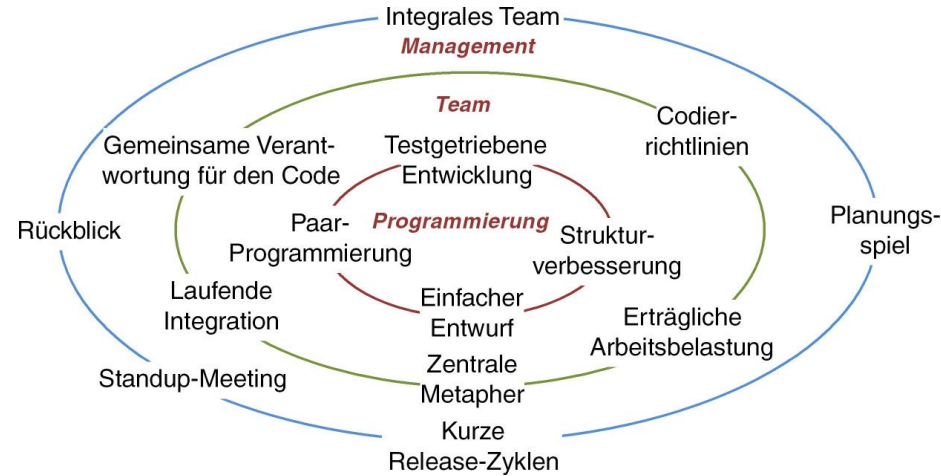
- Sie sind **iterativ** angelegt. Die **Zyklendauer** liegt zwischen 2 Wochen und einigen Monaten
- Die Arbeiten werden in **kleinen Gruppen** (6-8 Personen) durchgeführt
- Sie lehnen die Idee einer umfangreichen Dokumentation ab (zumindest mit Einschränkungen)
- Die **Kunden sind sehr wichtig** und deren Präsenz wird verlangt/erwünscht
- Dogmatische Regelungen werden abgelehnt

Agile vs. “Klassische” Modelle

	Bisheriger Ansatz	Agiler Ansatz
Ständige Mitwirkung des Kunden	Unwahrscheinlich	Kritischer Erfolgsfaktor
Etwas Nützliches wird geliefert	Erst nach einiger Zeit	Mindestens alle sechs Wochen
Das Richtige entwickeln durch	Langes Spezifizieren, Vorausdenken	Kern entwickeln, zeigen, verbessern
Nötige Disziplin	Formal, wenig	Informell, viel
Änderungen	Erzeugen Widerstand	Werden erwartet und toleriert
Kommunikation	Über Dokumente	Zwischen Menschen
Vorsorge für Änderungen	Durch Versuch der Vorausplanung	Durch „flexibel bleiben“

EXtreme Programming

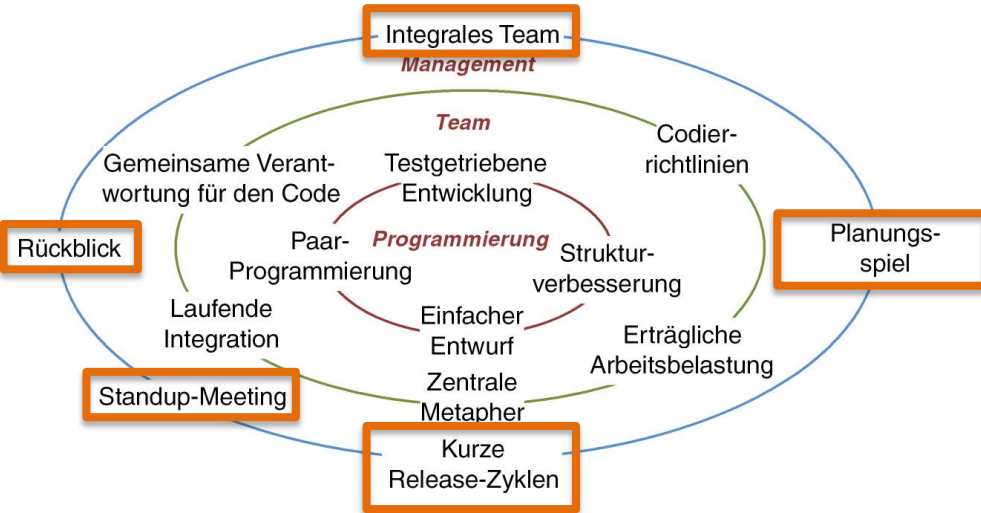
Charakteristika



- **Werte und Prinzipien:** Kommunikation, Feedback, Einfachheit, Mut zur Entscheidung
- **Konzepte:** Managementkonzepte, Teamkonzepte, Programmierkonzepte
- **Rollen:** Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss

EXtreme Programming

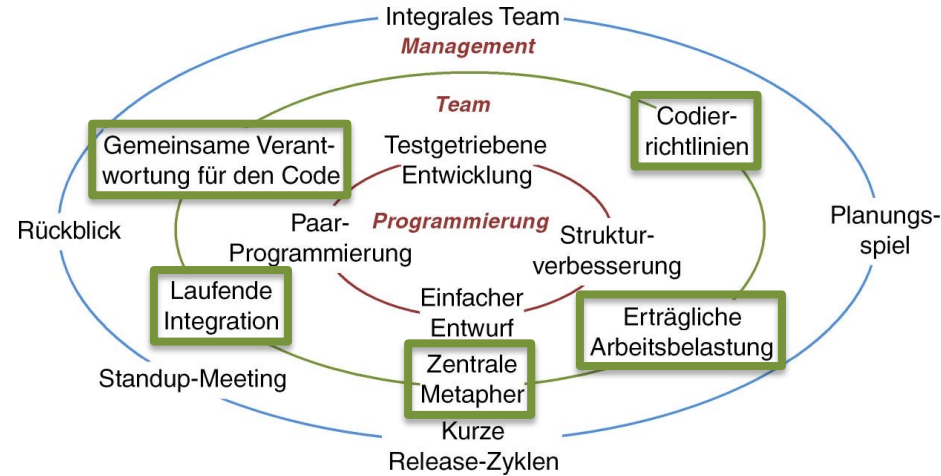
Charakteristika



- **Werte und Prinzipien:** Kommunikation, Feedback, Einfachheit, Mut zur Entscheidung
- **Konzepte:** Managementkonzepte, Teamkonzepte, Programmierkonzepte
- **Rollen:** Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss

EXtreme Programming

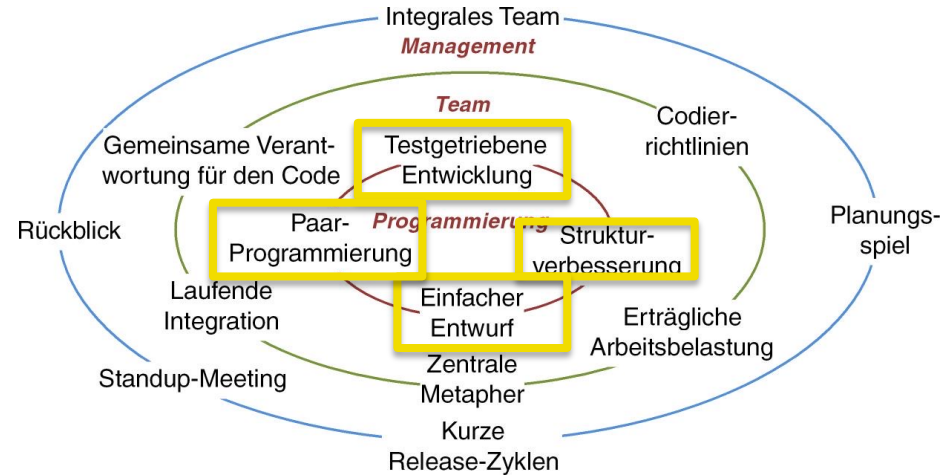
Charakteristika



- **Werte und Prinzipien:** Kommunikation, Feedback, Einfachheit, Mut zur Entscheidung
- **Konzepte:** Managementkonzepte, Teamkonzepte, Programmierkonzepte
- **Rollen:** Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss

EXtreme Programming

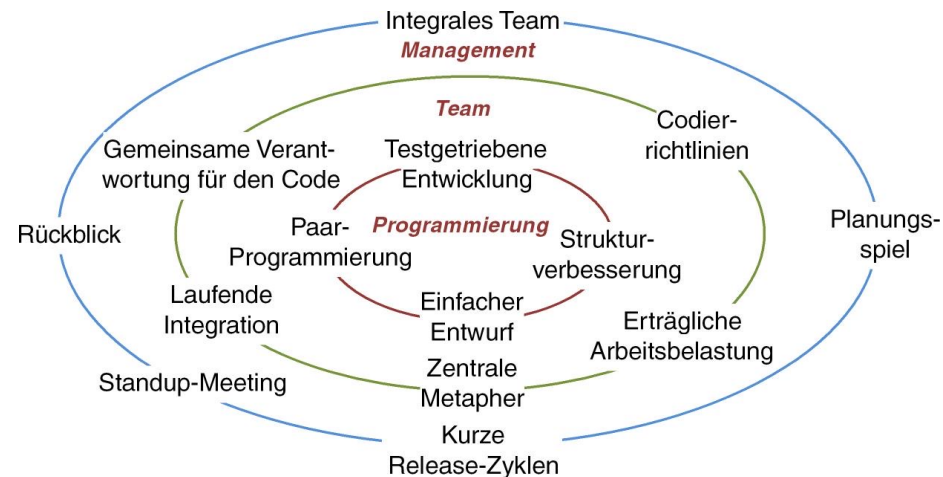
Charakteristika



- **Werte und Prinzipien:** Kommunikation, Feedback, Einfachheit, Mut zur Entscheidung
- **Konzepte:** Managementkonzepte, Teamkonzepte, Programmierkonzepte
- **Rollen:** Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss

EXtreme Programming

Charakteristika

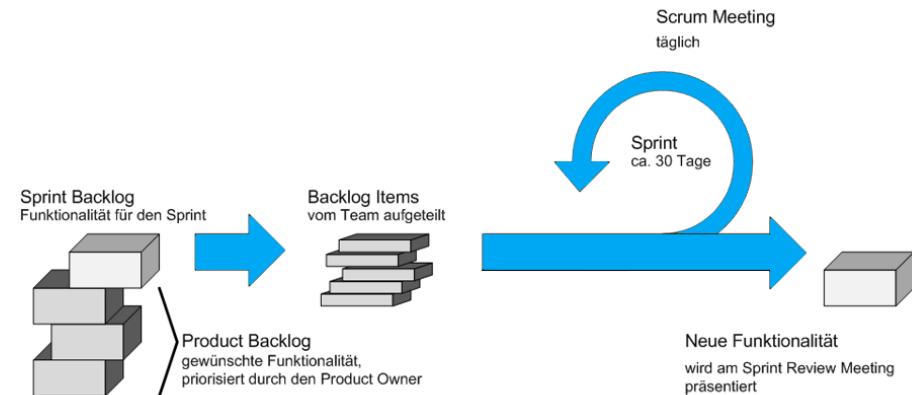


- **Werte und Prinzipien:** Kommunikation, Feedback, Einfachheit, Mut zur Entscheidung
- **Konzepte:** Managementkonzepte, Teamkonzepte, Programmierkonzepte
- **Rollen:** Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss

SCRUM

Charakteristika

- Keine strenge Arbeitsteilung, sondern kooperative Zusammenarbeit
- Selbst organisierendes Team
- Erfolg und Misserfolg wird kollektiv verantwortet
- Terminologie aus dem Rugby
 - „scrum“ Eroberung des Balles
- „sprint“ mit dem Ball um Raum zu gewinnen
- Scrum ist Rollen-basiert, iterativ und inkrementell



KANBAN

Charakteristika

- Orientiert sich an der selben Strategie, die Supermärkte nutzen, um ihre Regale zu füllen

- **Kanban-Board**

- Tool zur Visualisierung

- **Kanban-Karten**

- Jedes Aufgabenelement ist separate Karte
- Ermöglichen klare visuelle Darstellung des Workflows
- Stellen kritische Informationen zum jeweiligen Aufgabenfeld in den Vordergrund

Ready	Analysis		Construction		To Verify	Ready for Production
	In Progress 3	Done	In Progress 5	Done		
 	 	 	 		  	
						

Bewertung Agiler Prozesse

Anwendung

- Gut einsetzbar bei unklaren Zielen und sich ändernden Anforderungen
- innovative, zeitkritische Projekte

+ Verspricht verbessertes Kosten/Nutzen Verhältnis
+ Verbesserte durchschnittliche Code-Qualität

- Das Ergebnis ist nicht vorhersagbar
- Qualitätseigenschaften können nicht garantiert werden
- Weniger Planung
- Kunde (mangelndes Fachwissen)
- Zeitfaktor Refactoring

Welches SW-Prozessmodell ist das BESTE?



Welches SW-Prozessmodell ist das BESTE?



Das „beste“ Modell hängt von der Organisation, dem Produkt, bzw. den Fähigkeiten der Entwickler ab (es gibt kein ideales Modell!)